

Reviewer Pack — Diophantine Approximation Error Inversely Proportional to Co...

Entry e98d1643eeaa · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 00:27:37 UTC

Diophantine Approximation Error Inversely Proportional to Communication Discrepancy in XOR-Functions

Entry ID: e98d1643eeaa **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 00:27:37 UTC

1. Conjecture

Field A (mathematical branch): Diophantine Approximation **Field B** (complexity object): Communication Complexity of XOR-Functions

Statement:

For an XOR-function $f: \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}$, let $D(f)$ denote its communication discrepancy and $\varepsilon(f)$ denote the minimal approximation error of its Fourier coefficients by rational numbers with denominator $\leq n^2$. Then $D(f) \geq \Omega(1/\varepsilon(f) \cdot \log n)$.

Rationale (proposer's reasoning):

Diophantine approximation quantifies how well real numbers can be approximated by rationals, while discrepancy measures imbalance in communication matrices. Linking them could reveal structural constraints on XOR-functions' Fourier spectra, potentially separating complexity classes via approximation-theoretic barriers.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c0f0659e97a9d634

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=1.9s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def xor_function(x, y):
    return x ^ y

def continued_fraction_approximation(rational, denominator_bound):
    p0, q0 = 0, 1
    p1, q1 = 1, 0
    a = rational.numerator // rational.denominator
    while True:
        if q1 > denominator_bound:
            return Fraction(p0 + p1, q0 + q1)
        p0, q0 = p1, q1
        p1, q1 = p1 * a + p0, q1 * a + q0
        a = (rational.numerator - p1) // q1

def fourier_coefficients(xor_function, n):
    coefficients = {}
    for x in range(2**n):
        for y in range(2**n):
            if x == 0 and y == 0:
                continue
            coeff = (1 / (2**n)) * sum(xor_function((x >> i) & 1, (y >> i) & 1) for i in range(n))
            coefficients[(x, y)] = coeff
    return coefficients

def max_abs_error(coefficients):
    return max(abs(coeff) for coeff in coefficients.values())

def communication_discrepancy(xor_function, n):
    f = fourier_coefficients(xor_function, n)
    D = 0
    for x in range(2**n):
        for y in range(2**n):
            if x == 0 and y == 0:
                continue
            D += abs(f[(x, y)] - f[(x + 1) % (2**n)][(y + 1) % (2**n)])
```

```

return D / ((2**n)**2)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10
    c = 1.0 # Constant for the lower bound
    xor_function = lambda x, y: x ^ y

    metric_name = "communication_discrepancy / (epsilon * log(n))"
    instances_tested = 30
    total_D_f = 0
    total_epsilon_f = 0

    for _ in range(instances_tested):
        D_f = communication_discrepancy(xor_function, n)
        epsilon_f = max_abs_error(fourier_coefficients(xor_function, n))
        if epsilon_f == 0:
            return {
                "metric_name": metric_name,
                "metric_value": None,
                "instances_tested": instances_tested,
                "conjecture_holds": False,
                "counterexample": "epsilon_f is zero"
            }
        total_D_f += D_f
        total_epsilon_f += epsilon_f

    mean_D_f = total_D_f / instances_tested
    mean_epsilon_f = total_epsilon_f / instances_tested
    metric_value = mean_D_f / (mean_epsilon_f * math.log(n))

    conjecture_holds = metric_value >= c / (math.log(n))
    counterexample = "" if conjecture_holds else "epsilon_f is too small"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_metric_value = sum(result["metric_value"] for result in results if result["metric_value"])
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")

```

```

elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"epsilon_f is too small\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9b764fd6.py", line 101, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9b764fd6.py", line 67, in run_trial
    D_f = communication_discrepancy(xor_function, n)
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9b764fd6.py", line 52, in communication_discrepancy
    D += abs(f[(x, y)] - f[(x + 1) % (2*n)][(y + 1) % (2*n)])
           ~~~~~
KeyError: 1

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with KeyError during computation of communication discrepancy, preventing validation of conjecture | next: Fix the KeyError in communication_discrepancy function by ensuring all (x,y) pairs are properly initialized in the XOR-function's domain

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111468
2	propose	ollama_remote	qwen3:8b	0	0	53408
3	preregistration	ollama_remote	qwen3:8b	0	0	24303
4	novelty	ollama_remote	qwen3:8b	0	0	22580
5	novelty	ollama_remote	qwen3:8b	0	0	18170
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14834
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11391
8	judge	ollama_remote	qwen3:8b	0	0	18412

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 274566 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/e98d1643eeaa.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e98d1643eeaa.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e98d1643eeaa.tar.gz (if generated)